Database Systems
Laboratory Worksheet 08
Year 3 – SE Batch

IT23264670

Contents

Exercise 01 1
Exercise 02 2
Exercise 03 3
Exercise 04 4
Exercise 05 6

Exercise 01

Write a PL/SQL to get the current price of a stock belongs to company ‘IBM’. Store the company name (‘IBM’) in a variable.

```
DECLARE
   v_company VARCHAR2(12) := 'IBM';  -- Store company name
   v_price   NUMBER(8,2);          -- To store the current price
BEGIN
   -- Retrieve current price for the given company
   SELECT current_price
   INTO v_price
   FROM stocks_tab
   WHERE company = v_company;

   -- Optional: Print the result
   DBMS_OUTPUT.PUT_LINE('Current price of ' || v_company || ' is $' || v_price);
```

```
END;
/
```

Exercise 02

Write a PL/SQL to display a message for a given stock of ‘IBM’ company according to the following criteria.
Current price < 45 - ‘Current price is very low !’
45<=Current price < 55 - ‘Current price is low !’
55<=Current price < 65 - ‘Current price is medium !’
65<=Current price < 75 - ‘Current price is medium high !’
75<=Current price - ‘Current price is high !’
Store the company name in a variable.

```
DECLARE
   v_company VARCHAR2(12) := 'IBM';  -- Store company name
   v_price   NUMBER(8,2);           -- To hold current price
BEGIN
   -- Retrieve current price for IBM
   SELECT current_price
   INTO v_price
   FROM stocks_tab
   WHERE company = v_company;

   -- Check price range and display message
   IF v_price < 45 THEN
      DBMS_OUTPUT.PUT_LINE('Current price is very low !');
   ELSIF v_price < 55 THEN
      DBMS_OUTPUT.PUT_LINE('Current price is low !');
   ELSIF v_price < 65 THEN
      DBMS_OUTPUT.PUT_LINE('Current price is medium !');
   ELSIF v_price < 75 THEN
      DBMS_OUTPUT.PUT_LINE('Current price is medium high !');
   ELSE
```

```
    DBMS_OUTPUT.PUT_LINE('Current price is high !');
  END IF;
END;

/
```

Exercise 03

Write three PL/SQLs to draw the following shape. Use all three types of loops in each PL/SQL

```
9 9 9 9 9 9 9 9 9
8 8 8 8 8 8 8 8
7 7 7 7 7 7 7
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

1. Using FOR Loop (Outer), FOR Loop (Inner)

```
BEGIN
  FOR i IN REVERSE 1..9 LOOP
    FOR j IN 1..i LOOP
      DBMS_OUTPUT.PUT(i || ' ');
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('');
  END LOOP;
END;
/
```

2. Using WHILE Loop (Outer), FOR Loop (Inner)

```
DECLARE
  i NUMBER := 9;
BEGIN
  WHILE i >= 1 LOOP
    FOR j IN 1..i LOOP
      DBMS_OUTPUT.PUT(i || ' ');
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('');
    i := i - 1;
```

```
  END LOOP;
END;
/
```

3. Using SIMPLE LOOP (Outer), WHILE Loop (Inner)

```
DECLARE
  i NUMBER := 9;
  j NUMBER;
BEGIN
  LOOP
    EXIT WHEN i < 1;
    j := 1;
    WHILE j <= i LOOP
      DBMS_OUTPUT.PUT(i || ' ');
      j := j + 1;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('');
    i := i - 1;
  END LOOP;
END;

/
```

Exercise 04

The companies have decided to offer a bonus to the loyal clients who have purchased stocks of their company. A client who has purchased stocks before 1st January 2002 gets an additional 50 stocks as bonus. Those who have purchased stocks before 1st January 2001 get 100 stocks and for the purchases done before 1st January 2000 will be given 150 stocks as bonus.

Write a PL/SQL block to update the purchase table. Increase the quantities as given above. You must use a cursor for loop for this.

```
BEGIN
  -- Cursor FOR loop over all investments
  FOR rec IN (
    SELECT c.firstname,
```

```
        c.lastname,
        i.company,
        i.buy_date,
        i.qty,
        VALUE(i) AS investment_row,
        REF(c) AS client_ref
    FROM clients_tab c,
       TABLE(c.investments) i
) LOOP

    -- Variable to hold bonus
    DECLARE
      bonus NUMBER := 0;
    BEGIN
      -- Determine bonus based on purchase date
      IF rec.buy_date < TO_DATE('01-JAN-2000', 'DD-MON-YYYY') THEN
        bonus := 150;
      ELSIF rec.buy_date < TO_DATE('01-JAN-2001', 'DD-MON-YYYY') THEN
        bonus := 100;
      ELSIF rec.buy_date < TO_DATE('01-JAN-2002', 'DD-MON-YYYY') THEN
        bonus := 50;
      ELSE
        bonus := 0;
      END IF;

      -- Only update if bonus > 0
      IF bonus > 0 THEN
        -- Update the qty in the nested table
        UPDATE TABLE(
          SELECT c.investments
          FROM clients_tab c
          WHERE REF(c) = rec.client_ref
```

```
            )
            SET qty = qty + bonus
            WHERE company = rec.company
              AND buy_date = rec.buy_date
              AND buy_price = rec.buy_price;
        END IF;
    END;
  END LOOP;

  -- Optional: Confirm update
  DBMS_OUTPUT.PUT_LINE('Bonus stocks added successfully!');
END;

/
```

Exercise 05

Write a PL/SQL block with an explicit cursor to perform the same set of actions mentioned in exercise 04. You must use an explicit cursor and a while loop.

```
DECLARE
  -- Declare the explicit cursor
  CURSOR investment_cursor IS
    SELECT
      c.firstname,
      c.lastname,
      i.company,
      i.buy_date,
      i.qty,
      i.buy_price,
      REF(c) AS client_ref
    FROM clients_tab c,
       TABLE(c.investments) i;

  -- Record to hold each row fetched from the cursor
```

```
    rec investment_cursor%ROWTYPE;

    -- Variable for bonus calculation
    v_bonus NUMBER := 0;

BEGIN
    -- Open the explicit cursor
    OPEN investment_cursor;

    -- Fetch the first row
    FETCH investment_cursor INTO rec;

    -- WHILE loop: continue while a row was found
    WHILE investment_cursor%FOUND LOOP

        -- Determine bonus based on buy_date
        IF rec.buy_date < TO_DATE('01-JAN-2000', 'DD-MON-YYYY') THEN
            v_bonus := 150;
        ELSIF rec.buy_date < TO_DATE('01-JAN-2001', 'DD-MON-YYYY') THEN
            v_bonus := 100;
        ELSIF rec.buy_date < TO_DATE('01-JAN-2002', 'DD-MON-YYYY') THEN
            v_bonus := 50;
        ELSE
            v_bonus := 0;
        END IF;

        -- Only update if bonus > 0
        IF v_bonus > 0 THEN
            UPDATE TABLE(
                SELECT c.investments
                FROM clients_tab c
                WHERE REF(c) = rec.client_ref
```

```
        )
        SET qty = qty + v_bonus
        WHERE company = rec.company
          AND buy_date = rec.buy_date
          AND buy_price = rec.buy_price;
      END IF;

      -- Fetch the next row
      FETCH investment_cursor INTO rec;
    END LOOP;

    -- Close the cursor
    CLOSE investment_cursor;

    -- Optional confirmation
    DBMS_OUTPUT.PUT_LINE('Bonus stocks added using explicit cursor and WHILE loop!');
END;
/
```